A writable BTHome device, end to end

First working use case for `bthome-writable`: Home Assistant commands a Puck.js over a short GATT connection, and the device confirms by advertising — with its own light sensor measuring that the command had a physical effect.

Recorded 8 September 2026. Provisional: this is one device, one receiver, one afternoon.

---

1. What was set up

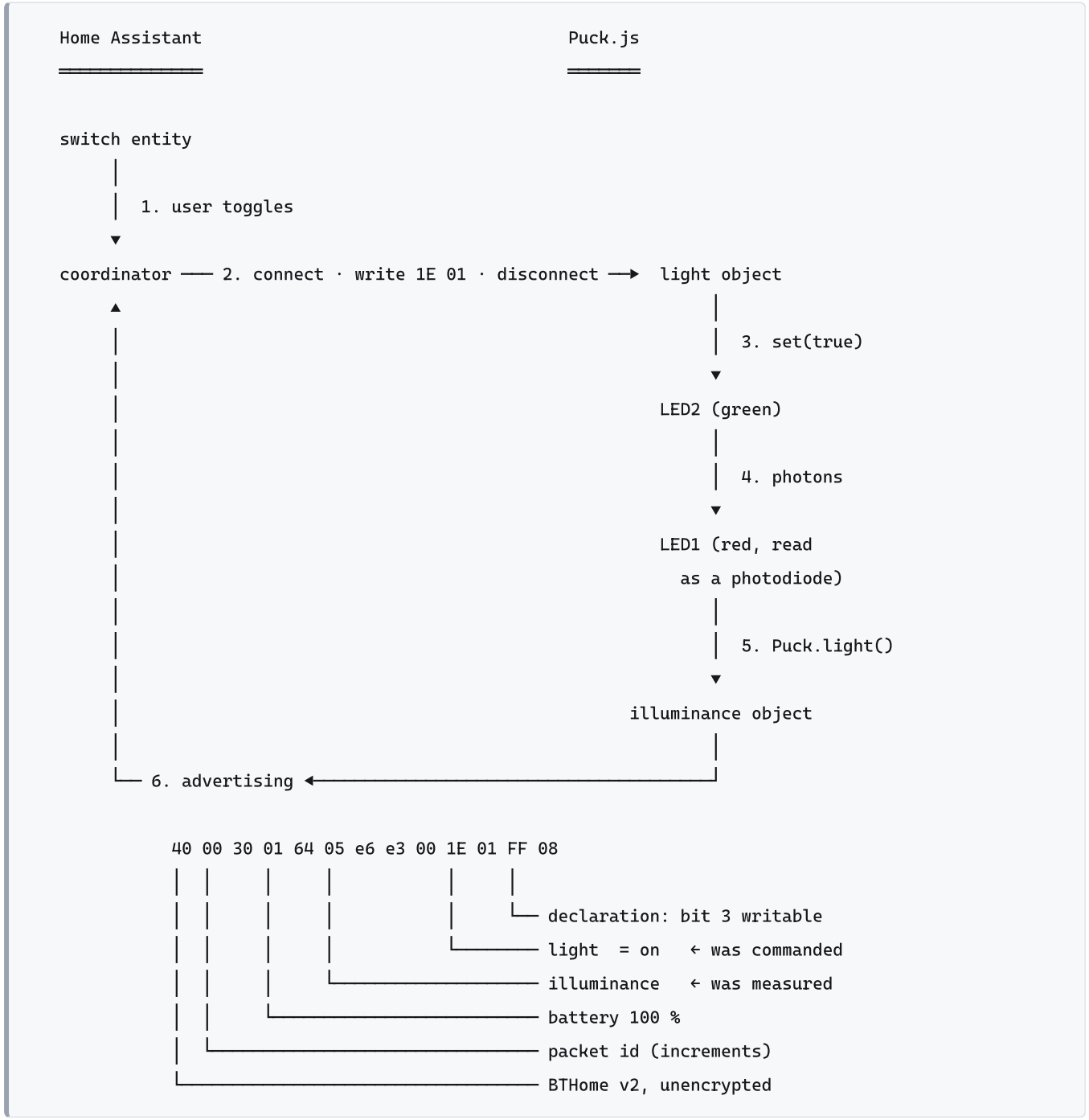
Device	Puck.js, Espruino 2v27, static address `C8:80:32:AD:F7:B9`, CR2032 at 3.00 V
Device code	`espruino/examples/light-loop.js`, in RAM, refresh interval 2 s
Receiver	Home Assistant 2026.7.4 (HAOS) on a Raspberry Pi 3, built-in `bcm43438` adapter
Integration	`bthome_writable`, deployed to `config/custom_components/`
Reference host	Windows workstation with its own adapter, for independent capture

No Bluetooth proxy was involved: all three ESPHome proxies configured on that Home Assistant were offline, so every measurement went through the Pi's own single adapter. That turns out to matter a great deal (§5).

---

2. The loop

The point of this particular device is that the return path is a **measurement, not an echo**. `Puck.light()` reads through the red LED, so the green one is the actuator and the two never share a part.



One packet carries both the state that was asked for and an independent measurement of whether it happened. A device that merely stored a value and echoed it back could not produce the second one.

---

3. What the device advertises, and what it accepts

The advertised packet is exactly what the specification predicts, declaration last, and 13 bytes against a 24-byte budget (`spec/PROTOCOL.md` §2.3).

A write is the concatenation of every writable object in packet order (§4.2). Here there is one, so a write is two bytes: `1E 01` or `1E 00`.

Malformed writes were exercised against the device directly (`python -m tools.reject_matrix`). All four were rejected with the expected code, and **the advertised state was unchanged afterwards** — nothing was partially applied:

Write	Rejected as	Why it matters
`1F01`	`objectid_mismatch`	The receiver is writing against a stale layout
`1E`	`truncated`	A value cut short
(empty)	`truncated`	
`1E01FF02`	`trailing_bytes`	This is what a **replayed advertisement** looks like: a valid prefix followed by the declaration. A permissive parser would apply the prefix

---

4. Closing the loop

Commanding the light and reading the device's own illuminance object back:

Commanded	Advertised packet	`light`	Illuminance
off	`4000 17 0164 05f92c00 1E00 FF08`	0	115.1
on	`4000 24 0164 05e6e300 1E01 FF08`	1	**583.4**
off	`4000 30 0164 05e62c00 1E00 FF08`	0	114.9

5.1×, and the two dark readings agree to within 0.2. The commanded LED is measurably lighting the sensor.

The units are not lux — `Puck.light()` is uncalibrated and the example scales it arbitrarily. That is deliberate: the figure only has to *move* with the command, and it does.

This is what answers the question the software could not. Everything else in the project is satisfied by a device that stores what it was told; this is not.

---

5. Timings

Two different things get called "latency" here, and conflating them is how the first version of the receiver got the confirmation model wrong.

5.1 What was measured

End-to-end confirmation — click to the advertisement that proves it. Measured on the Pi, by polling the entity through the REST API at 250 ms while the receiver's window was still (wrongly) short enough to expire first:

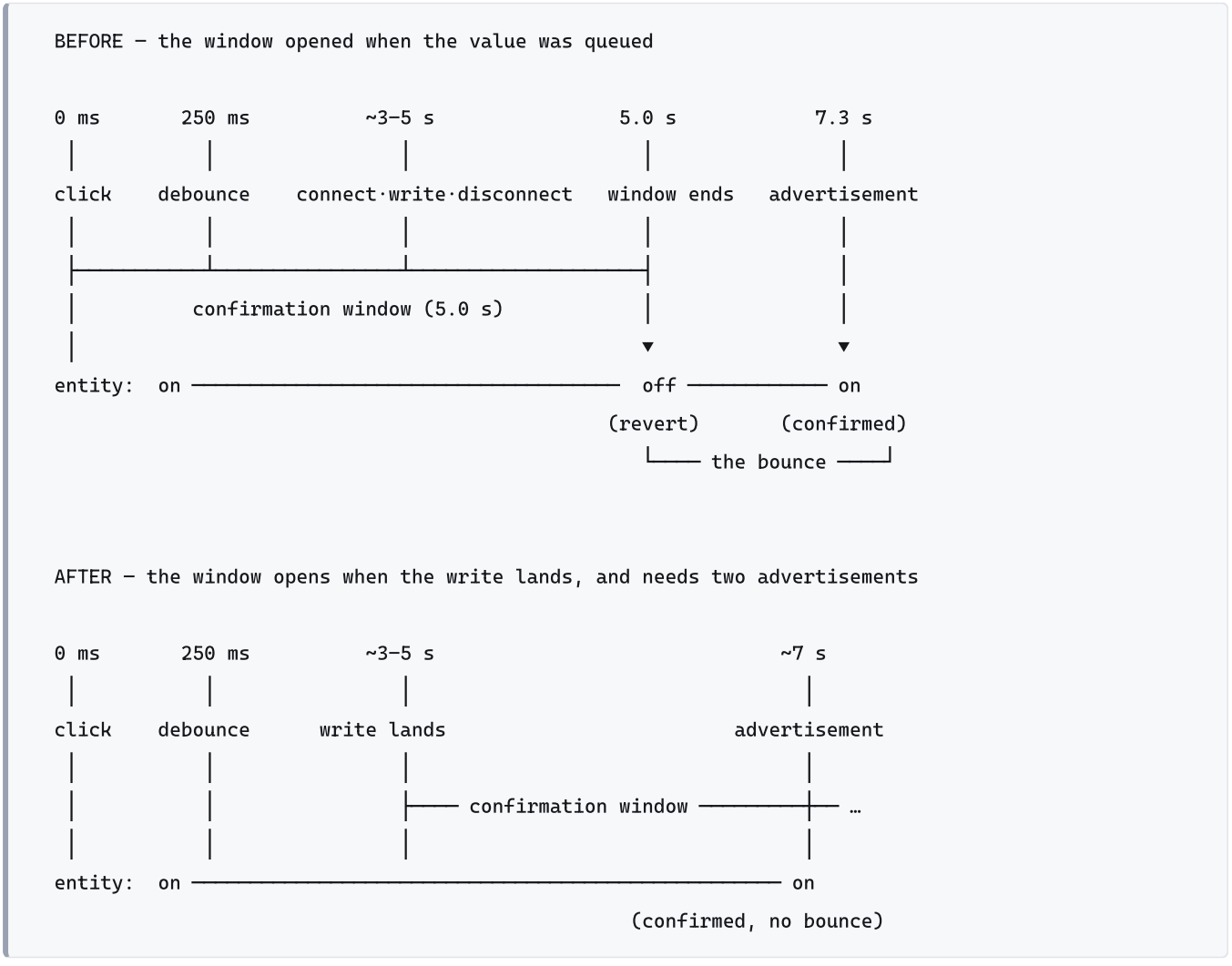
Toggle	Optimistic update	Reverted at	Confirmed at
on	500 ms	5 343 ms	**7 312 ms**
off	485 ms	5 407 ms	**6 391 ms**

So a full round trip on this hardware took **6.4 to 7.3 seconds**. Against a 5-second window, that is why every toggle bounced.

The write path alone, from the Windows host — connect, write, disconnect: **4.6 s, 5.6 s, 6.7 s, 10.7 s**. Wide, and dominated by connection setup rather than by the write.

Optimistic update: 469–500 ms, consistently. That is the 250 ms coalescing debounce plus REST round trips plus a 250 ms polling granularity — it measures the receiver's responsiveness, not the device.

5.2 Before and after



After the change, **four consecutive toggles from the Home Assistant UI settled with no spurious transition** (`decisions.md` D-010, D-011).

5.3 What was *not* measured

The confirmation latency after the fix. Once the entity stops bouncing there is no observable transition to time — only the absence of a revert, which bounds it below the window and no more. Measuring it properly needs the receiver to log the confirming advertisement, or a second adapter dedicated to scanning.

That is the main gap in this report.

5.4 The reason the numbers are so large

A host with one Bluetooth adapter cannot scan while it is connected. Every write therefore begins by deliberately blacking out the exact channel the confirmation must arrive on, and the adapter then takes seconds to resume.

Measured on the Windows host: **four advertisements caught in thirty seconds** from a device advertising every two, with complete silence for stretches after each connection. Several apparent device failures during this session turned out to be the host's radio; the Puck was advertising throughout, its packet id incrementing.

An ESPHome proxy, which does the connecting while the host keeps listening, should change these figures substantially. That remains untested.

---

6. What broke, and what it taught

Six defects surfaced. Only the first two are interesting; all six are the kind that no amount of software testing was going to find.

1. **The confirmation window started at the wrong moment** (D-010). It opened when the entity queued its value, so the debounce, the connection, the write and the disconnect were all inside a window meant to measure only the device's advertising refresh. Mocked tests cannot catch this: with a mocked transport the two moments coincide. Only a radio separates them.

2. **Reverting on no evidence** (D-011). Even with the window correctly placed, it was a plain timer — so the receiver could write an entity off having heard nothing at all from the device, which on a single-adapter host is the normal case. An unconfirmed value is now reverted only once the window has elapsed *and* two advertisements have arrived since the write.

3. **An advertising overflow**, caught by arithmetic before flashing: the module advertised its 128-bit service UUID, 18 bytes of a 31-byte payload, which with the Flags structure and the BTHome service data does not fit. It also buys nothing — the receiver finds the device by its BTHome service data and discovers the service after connecting.

4. **A constant that was silently undefined on the device.** The Espruino console evaluates a pasted top-level statement as soon as a line ends outside any bracket, so a wrapped `var SERVICE_DATA_BUDGET = …` left the constant undefined — which quietly disabled the capacity check, everything comparing false against `undefined`.

5. **The packet was built once and never again.** Sensor values froze at their boot readings and the packet id stopped moving, which is what a receiver uses to tell a fresh advertisement from a repeat. Spotted by noticing four consecutive captures all carrying packet id 1.

6. **A name collision with the upstream module.** Its `raw` type means "emit these bytes verbatim" and is *not* BTHome's length-prefixed raw object `0x54`; this module had listed it as length-prefixed, which would have put a phantom length byte in the write layout of any writable raw entry.

---

7. Provisional conclusion

The mechanism works. A device declares writability in ordinary BTHome advertising; a receiver discovers it, writes in BTHome's own format over one GATT characteristic, and the refreshed advertising confirms it. No new data format, no acknowledgement protocol, no bonding. The declaration rides in the service data without disturbing any existing BTHome receiver.

Three things are more solid than expected. Positional addressing survives a device being reflashed with a different layout: the stale entity went *unavailable* rather than writing to the wrong object. Device merging works — the switch landed on the existing card, inheriting its name and area. And the `not_supported` abort held on real data: on a box with eight BTHome devices configured, exactly one was offered.

One thing is harder than expected: the confirmation model. "The device refreshes its advertising, the receiver sees it" is a clean idea that hides a race with the receiver's own radio. On a single-adapter host the receiver is deaf for several seconds *because* it wrote. Both defects worth calling defects came from that, and the design now rests on counting advertisements rather than on elapsed time. The specification says only that a receiver must revert if no confirming advertisement arrives "within its confirmation window" — correct, but the obvious reading of a window is a duration, and that reading is wrong. Worth a sentence in §6 when it is next revised.

What this does not establish. One device, one receiver, one radio environment. Nothing here exercises an ESPHome proxy, multiple writable objects on one device, a device on a battery-saving advertising interval of tens of seconds, several receivers, or any of the encryption in §5 of the protocol — for which test vectors exist but no implementation. The confirmation latency after the fix is unmeasured. And the device ran from RAM: a power cycle wipes it.

---

8. Reproducing this

```
python -m tools.build_espruino_bundle
python -m tools.espruino_upload --address <mac> espruino/dist/light-loop-standalone.min.js
python -m tools.closed_loop     --address <mac>      # §4
python -m tools.reject_matrix   --address <mac>      # §3
```

Home Assistant side: copy `ha/custom_components/bthome_writable/` into `config/custom_components/`, restart, and add the device — it appears on its own, or under *Add integration → BTHome Writable*.

Full findings and reasoning: `spec/decisions.md` (D-001 to D-011). Open questions for the Espruino discussion: `spec/for-gordon.md`.